Lecture 9: Scheduling Algos

Professor G. Sandoval

Some slides adapted by G. Sandoval for CS3224 from Brendan Dolan-Gavitt

My understanding of XV6 Scheduling mechanism is best described by:

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Scheduling Algorithms

- Intro
- Scheduling in Batch Systems
- Scheduling in Interactive Systems
- Scheduling in Real-Time Systems
- Scheduling in XV6

Scheduling Algos

- Whenever # processes > #cpus need to use scheduling algorithm.
- Why?
  - Many processes could be in the ready state.

Metrics

- **Throughput** – jobs completed / time interval

- **Turnaround** – average time between when a job is submitted and when it completes

- **Response time** – time between when a user issues a command and gets the result

Tradeoffs

- Improving on one metric can hurt another
- For example:
  - We want to improve *throughput*, so we decide to only schedule short jobs
  - But now longer jobs never get run, so their *turnaround time* is effectively **infinite**

Scheduling Algorithm Goals

- All Systems:
  - **Fairness** – Giving each process a fair share of the CPU
  - **Policy Enforcement** – seeing that stated policy is carried out
  - **Balance** – keeping all parts of the system busy

Scheduling Algorithm Goals

Batch Systems:

- **Throughput** – maximize jobs per hour
- **Turnaround time** – minimize time between submission and termination
- **CPU utilization** – keep the CPU busy all the time

Scheduling Algorithm Goals

Interactive Systems:

- **Response Time** – respond to requests quickly
- **Proportionality** – meet user expectations

Scheduling Algorithm Goals

Real-Time systems:

- **Meeting Deadlines** – avoid loosing data
- **Predictability** – avoid quality degradation

Scheduling Algorithms

- Intro
- Scheduling in Batch Systems
- Scheduling in Interactive Systems
- Scheduling in Real-Time Systems
- Scheduling in XV6

First Come First Served

- Single queue of ready processes
- The process at the head of the queue runs **as long as it likes** or until it blocks (no quantum)
- After it runs, you add it to the back of the queue and let the next one in line run

FCFS

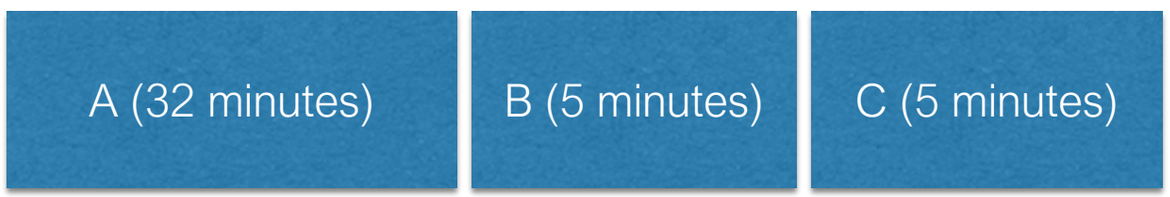
- Very easy to program! Can just use a linked list.
- But I/O may suffer
  - CPU-bound process can take up lots of time
  - But I/O bound process will have to block and then wait until it gets back to the head of the queue before it can issue another I/O

Analyzing First Come First Served

- To measure turnaround time:
  - $\text{Time}_{(\text{completed})} - \text{Time}_{(\text{submitted})}$
- Average Turnaround time:
- $\sum (\text{Time}_{(\text{completed})} - \text{Time}_{(\text{submitted})}) / N$

Analyzing First Come First Served

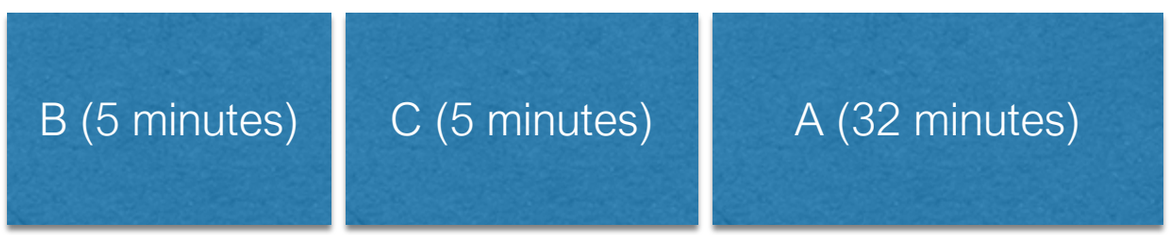
- Turnaround time depends on order we pick jobs
  Assuming jobs arrive at time 0:



Turnaround time: (32 + 37 + 42) / 3 = 37 minutes

Analyzing First Come First Served

- Turnaround time depends on order we pick jobs
  Assuming jobs arrive at time 0:



Turnaround time: (5 + 10 + 42) / 3 = 19 minutes

Shortest Job First

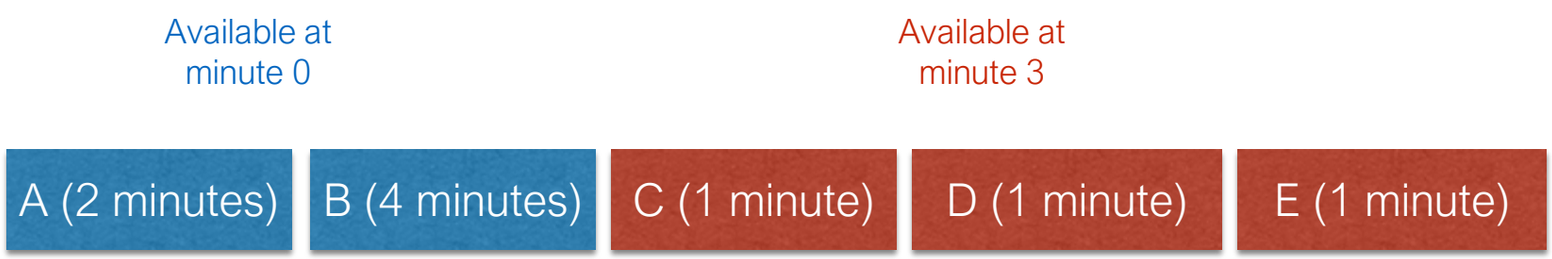
- Batch, non-preemptive
- Assumes we can predict how long each job will take to execute
- Always picks the job with the shortest time to execute to run

Analyzing Shortest Job First

- Using SJF provably minimizes turnaround time
- To see why, consider scheduling 3 jobs with runtimes a, b, and c
- Turnaround time
  $= (a + (a + b) + (a + b + c)) / 3$
  $= (3a + 2b + c) / 3$
- So to minimize turnaround time, we want a to be as small as possible (since it has the largest effect on the average)

Counterexample

- The optimality proof only applies when all jobs are available at time 0
- Suppose we have instead:



Then turnaround time is
(2 + 6 + (7 - 3) + (8 - 3) + (9 - 3)) / 5 = 4.6

- But if we run them in the order B, C, D, E, A, time is:
(4 + (5 - 3) + (6 - 3) + (7 - 3) + 9) / 5 = 4.4

Scheduling Algorithms

- Intro
- Scheduling in Batch Systems
- Scheduling in Interactive Systems
- Scheduling in Real-Time Systems

Interactive Scheduling

- In an interactive system, scheduling algorithms are generally *preemptive*
- Time is divided up into slices called *quanta*
- Each process runs for 1 *quantum* and then the scheduler runs again

Round Robin Scheduling

- Again, simple algorithm
- Run first process until its quantum is used up
- Move that process to the end and run the next process until its quantum is used up
- Simple, fair

Design Considerations with Round Robin

- The length of the quantum is typically determined by a hardware timer interval
- This is generally configurable
- So: how long should we make a quantum?

Design Considerations with Round Robin

- Context switching takes some amount of time (swap out CPU registers, change address space)
- Consider:
  - context switching = 1ms,
  - quantum = 4ms,
  - => 20% of time spent just switching

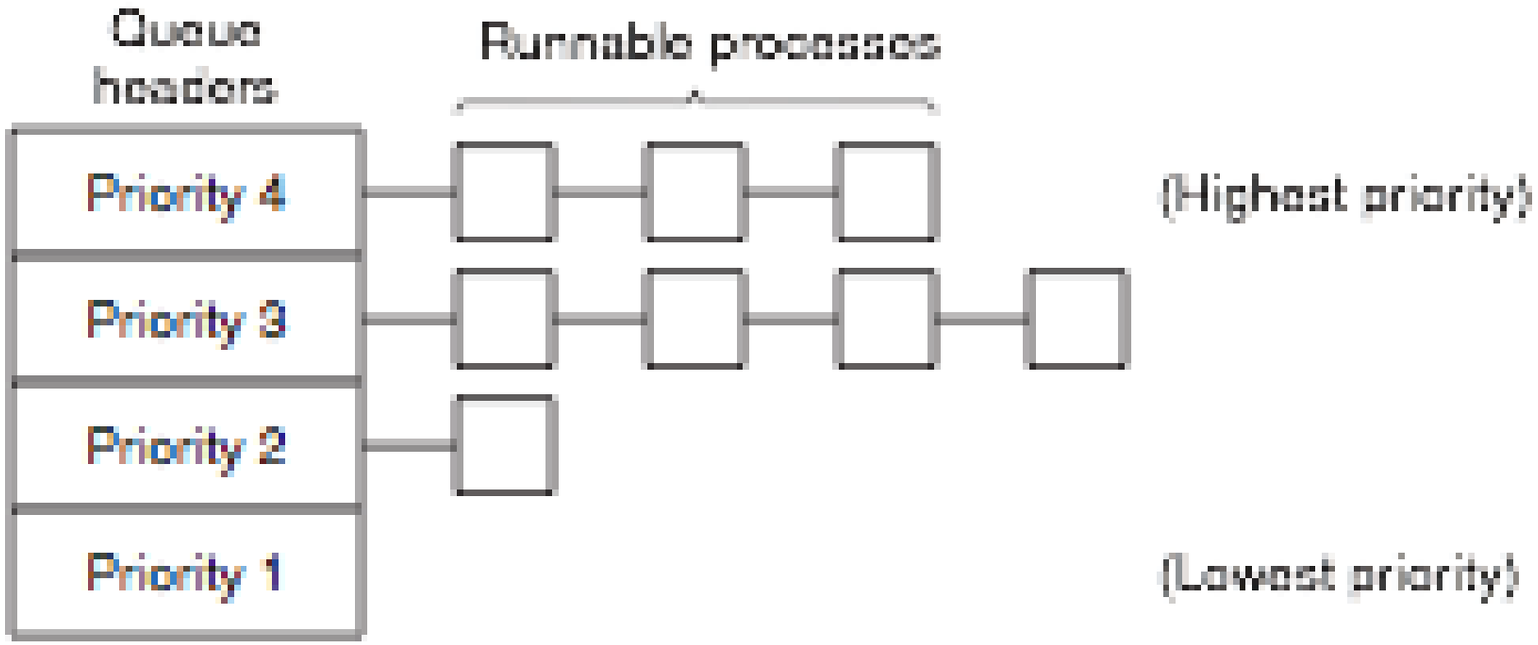
Design Considerations with Round Robin

- What if:
  - Large quantum = 100ms ?
  - => 5 seconds before a process gets to run on a system with 50 active processes
- Tuning this is a balancing act. Values around 10-50ms are common
- xv6 uses a 10ms quantum

Priorities

- Round robin scheduling assumes that every process has the same **priority**
- In reality, we may always want some processes to get scheduled before others
- Possible policies:
  - Safety-critical code should run first
  - Users who pay more should get higher priority
  - Interactive processes should get priority over daemons

Priority Scheduling



Dynamic Priority

- We can also adjust the priority of processes *dynamically*
- One nice way to use this is to dynamically give I/O bound processes more chances to run
- We can set the priority as a function of the fraction of the last quantum the process actually used:

  priority = 1 / f f = fraction of quantum used

- This would give *higher* priority to processes that used a smaller fraction of their quantum – i.e., processes that waited for I/O

Shortest Process Next

- *Latency* in an interactive system is analogous to *turnaround time* in a batch system
- Unfortunately in an interactive system we don't necessarily know how long a command will take
- But we can make *estimates,* and then update our estimates over time with real data

Process Aging

- Set some initial estimate T_0
- Run the process and measure to get T_1
- Fix *a*, the aging parameter; $0 \leq a \leq 1$
- After running a process, update the estimate as:
  $T_i = aT_{i-1} + (1-a)T_{i-2}$

Process Aging

- So with a = 1/2, the estimates become:
  - T_0,
  - $T_0/2 + T_1/2$,
  - $T_0/4 + T_1/4 + T_2/2$,
  - $T_0/8 + T_1/8 + T_2/4 + T_3/2$
- Over time, our initial estimate is weighted less and less and more recent events have greater weight

Remember:

$T_i = aT_{i-1} + (1-a)T_{i-2}$

Guaranteed Scheduling

- Guarantee to each of n processes that they will get 1/n about 1/n of the CPU
- As processes run, keep track of how much CPU time they have actually used
- Now we can schedule processes based on how "unfair" we have been to them up to this point
- Easy to say, hard to implement!

Lottery Scheduling

- We can get a probabilistic version of guaranteed scheduling that is much easier to implement
- Give each process a fixed number of *lottery tickets*
- When it comes time to schedule, pick a random number between 1 and the number of tickets
- Schedule the process that won the lottery

Lottery Scheduling

- We can give each process a proportion of the CPU by just giving it that proportion of the tickets
- We can also guarantee that every process *eventually* gets to run as long as it gets at least one ticket
- It's not a true guarantee, however – only a probabilistic one

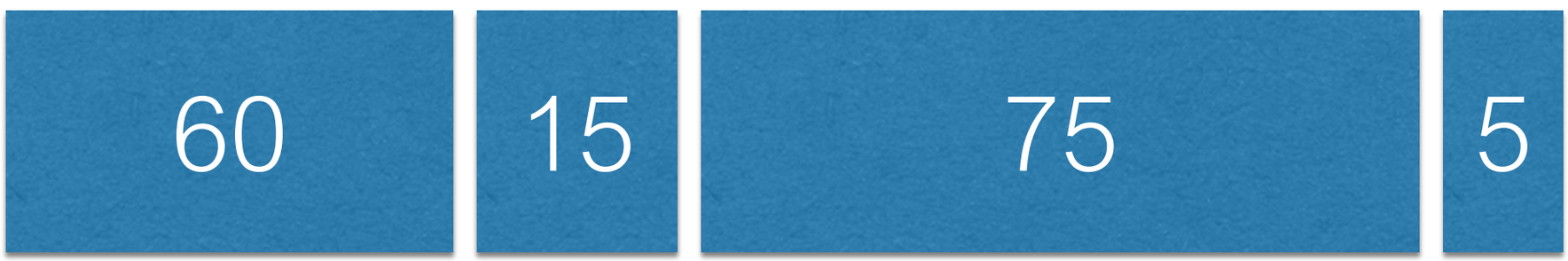
Lottery Scheduling

“All processes are equal, but some are more equal” – George Orwell

Implementing Lottery Scheduling

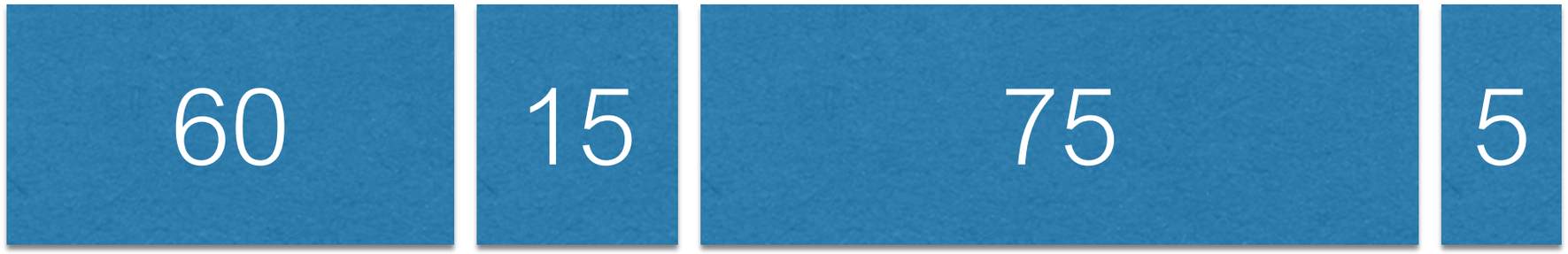
- Take our existing process control block (PCB) data structure and augment it with a **num_tickets** field
- At scheduling time:
  - Generate a random ticket number *winner*
  - Loop over processes, keeping a *counter*
  - If *counter* ≥ *winner* then pick that process
  - Otherwise, add the process's tickets to *counter* and continue

Lottery Scheduling

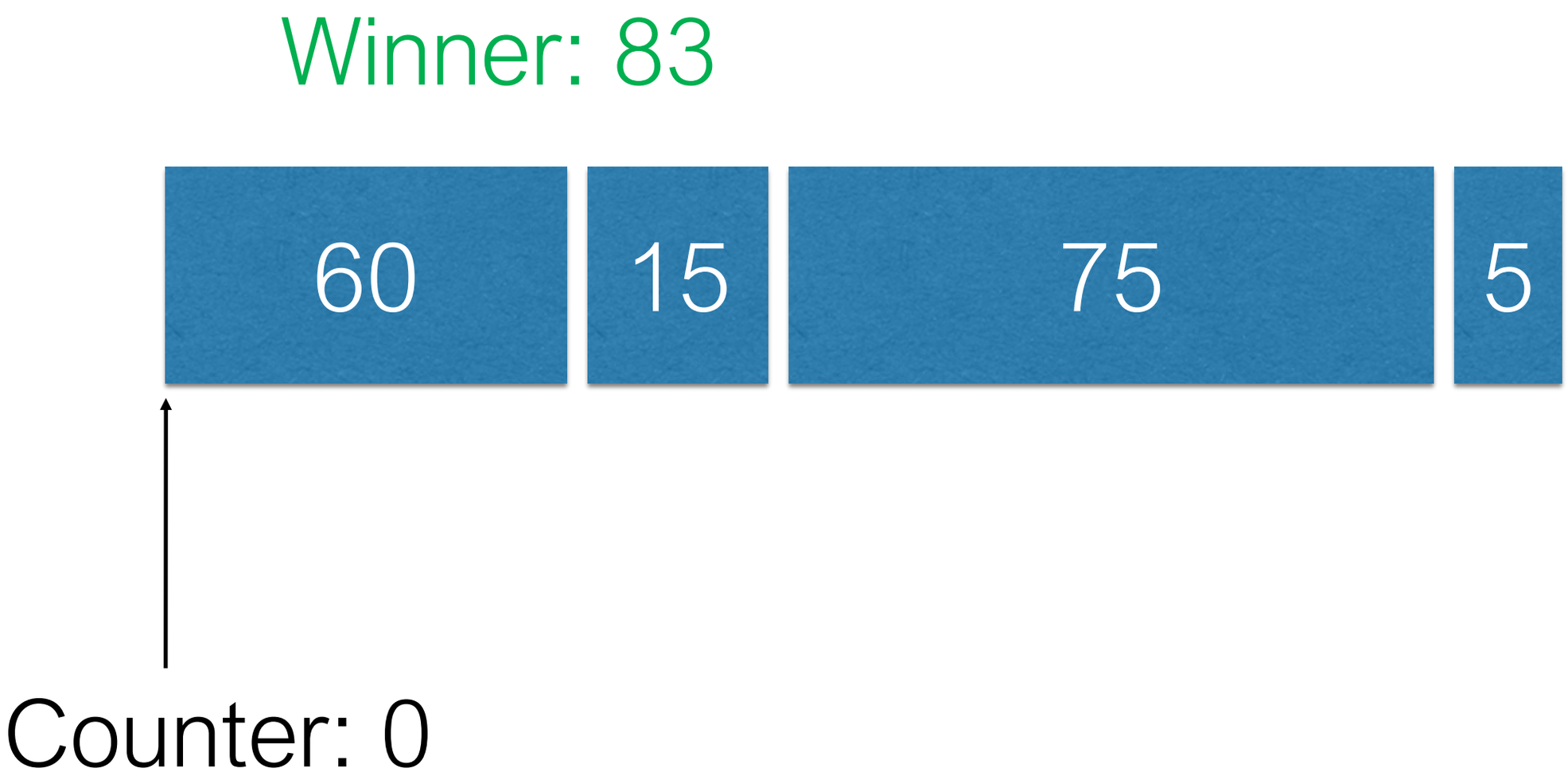


Lottery Scheduling

Winner: 83

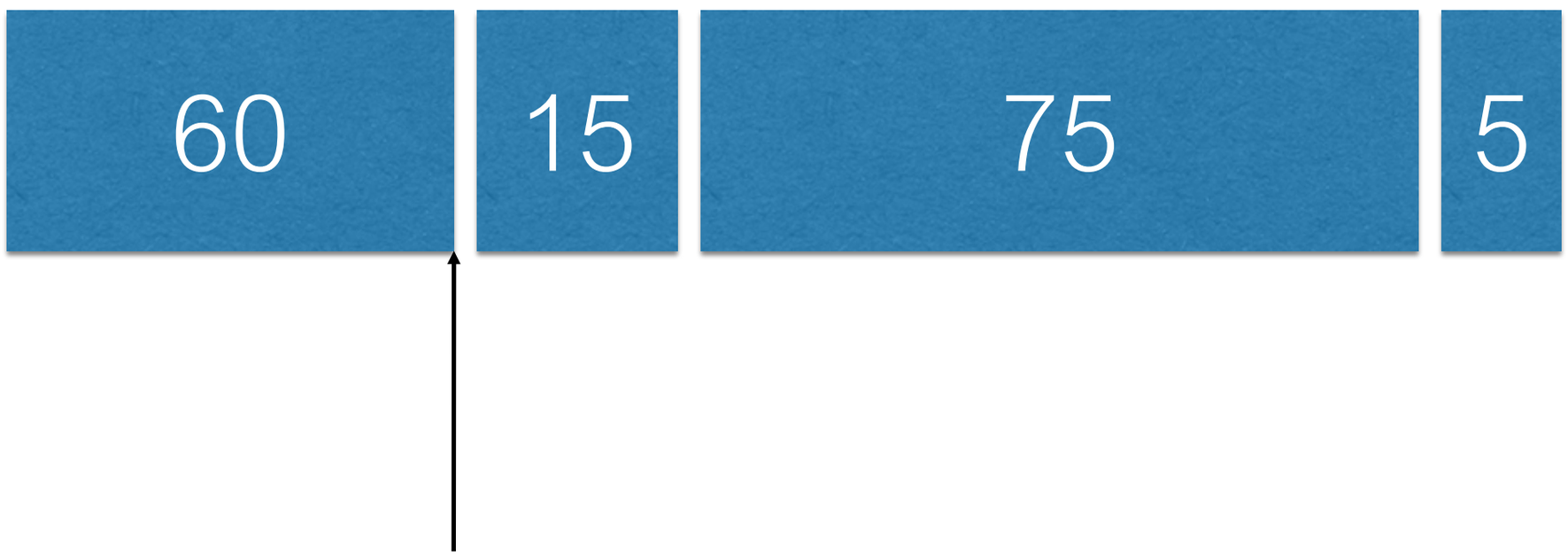


Lottery Scheduling



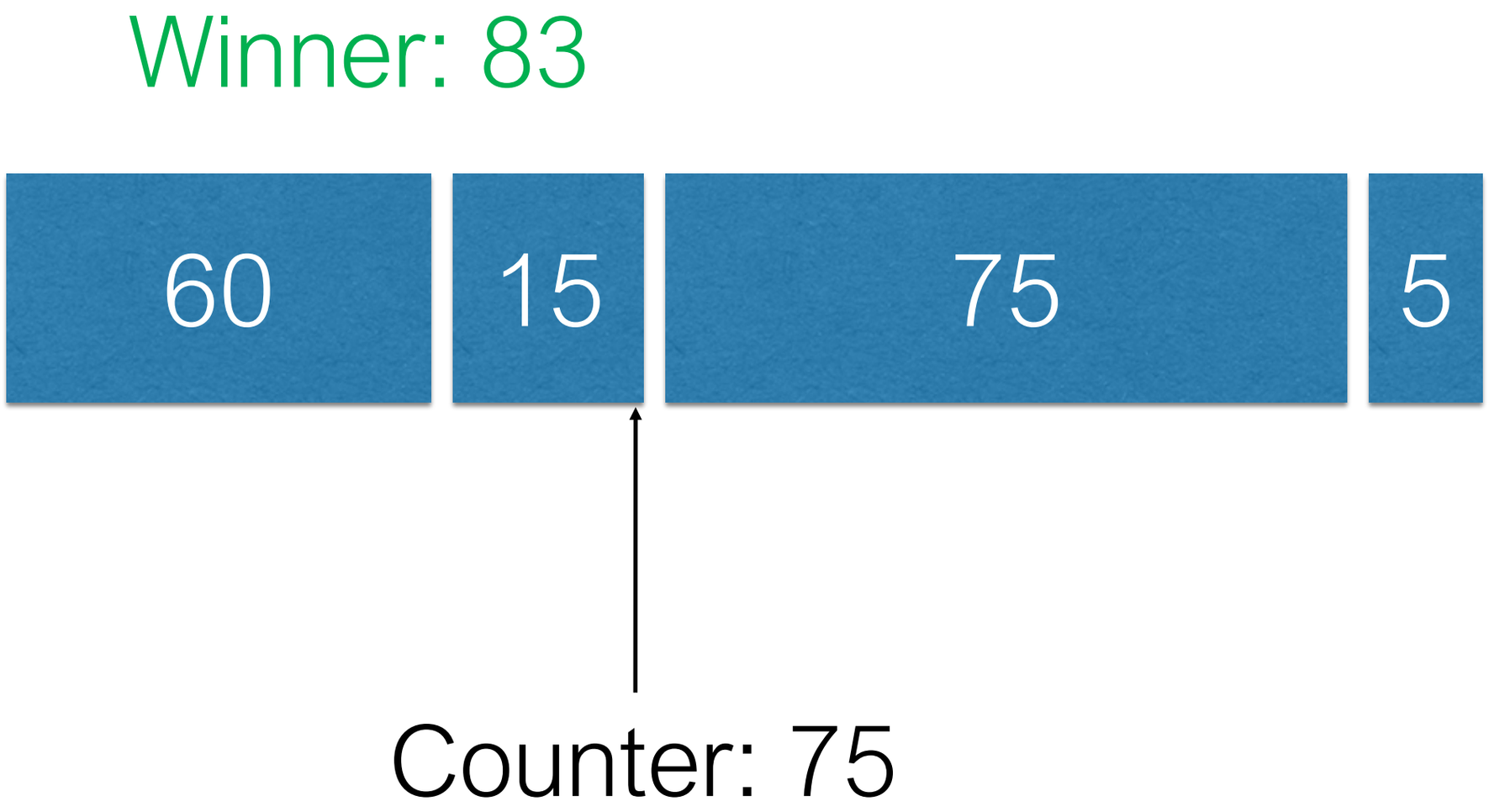
Lottery Scheduling

Winner: 83

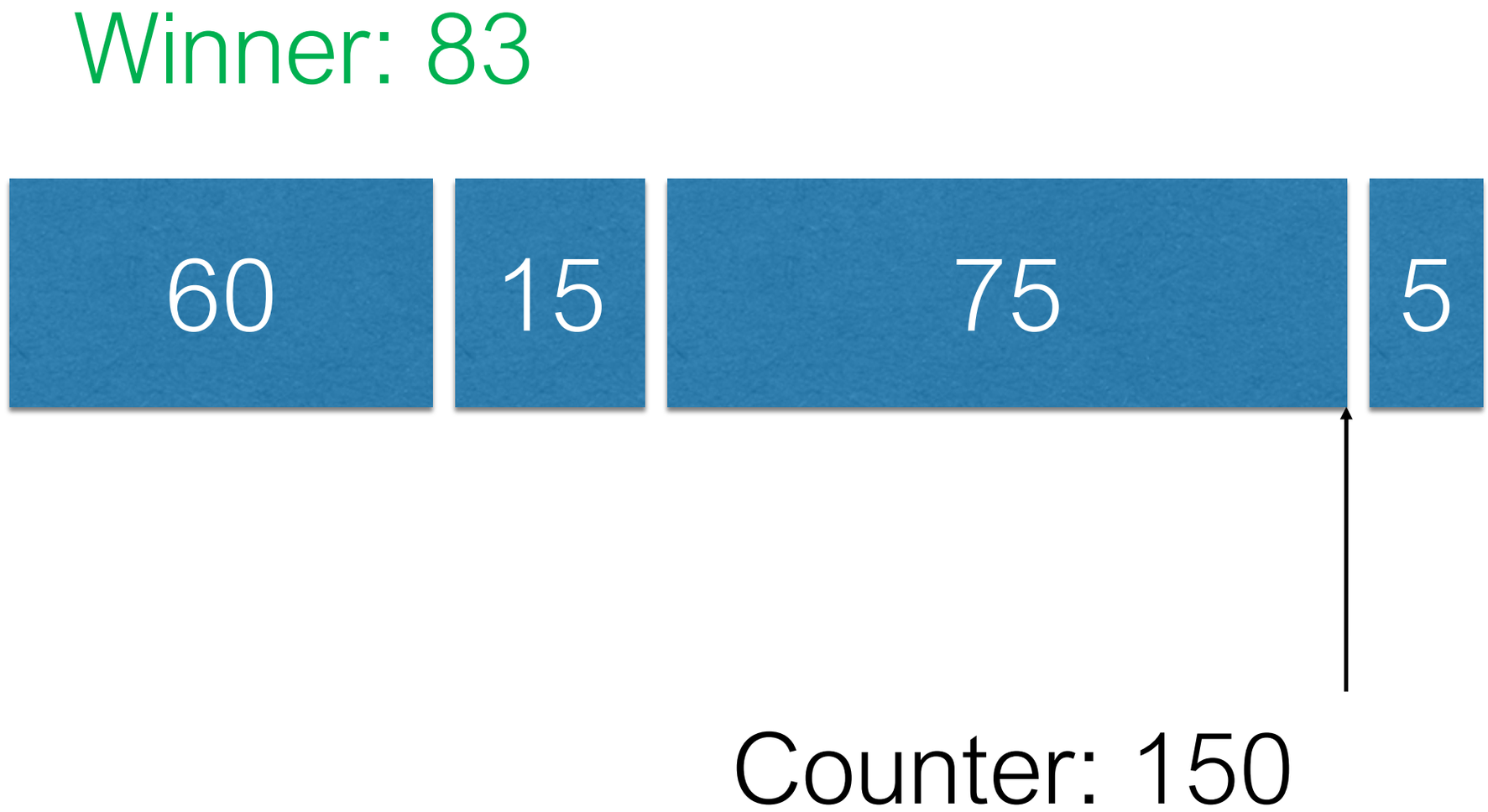


Counter: 60

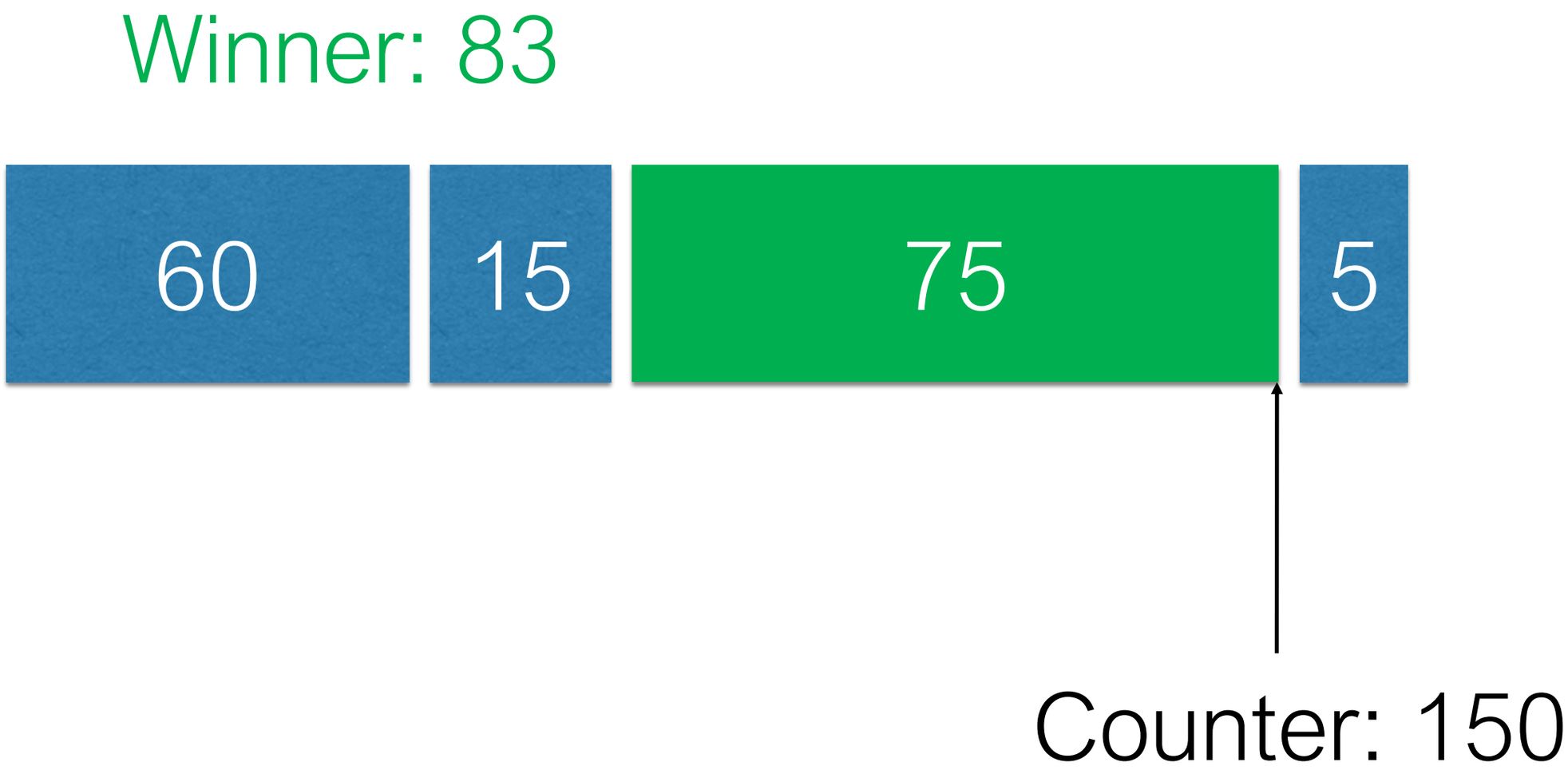
Lottery Scheduling



Lottery Scheduling



Lottery Scheduling

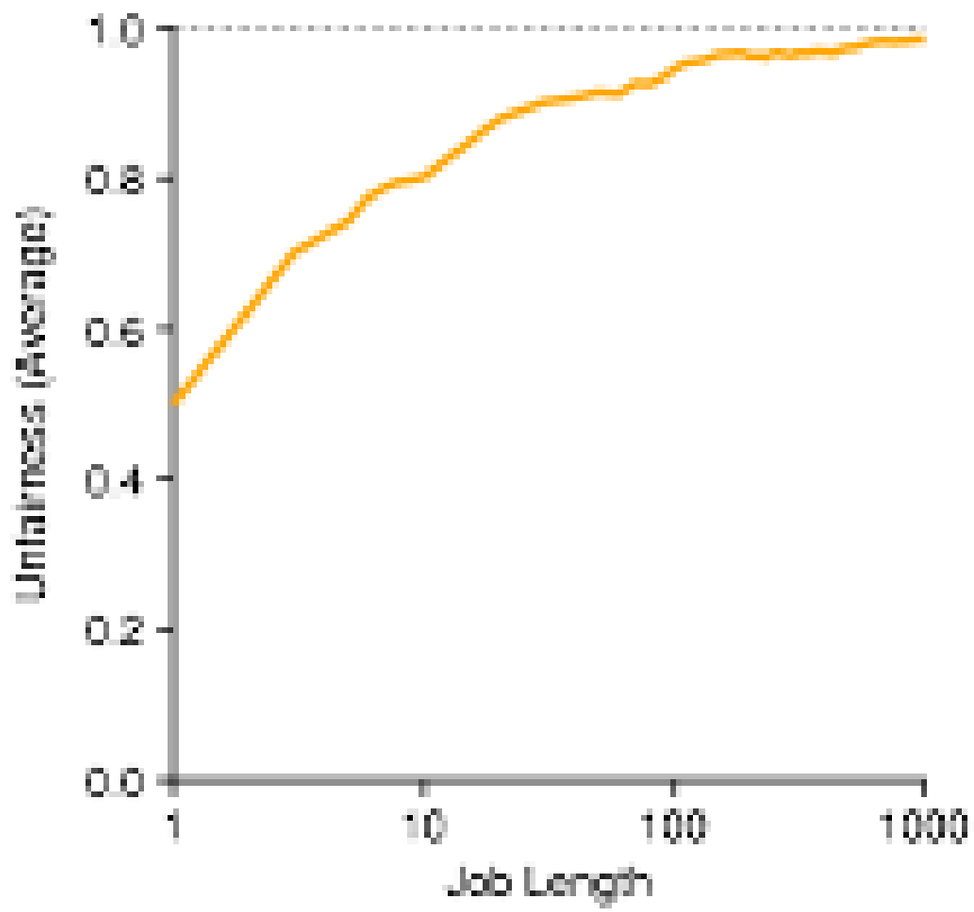


Choose a Lottery number

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Analyzing Lottery Scheduling



Scheduling Algorithms

- Intro
- Scheduling in Batch Systems
- Scheduling in Interactive Systems
- Scheduling in Real-Time Systems
- Scheduling in XV6

Real-Time Scheduling

- Real-time operating systems receive events and must respond to them within a fixed amount of time
  - **Multimedia systems** – if you don't process audio/video fast enough, you get pauses & skips
  - **Control systems** – autopilot, industrial manufacturing control
- Hard real time vs soft real time – how bad is it to miss a deadline?

Is it Schedulable?

- Events in real-time systems can be *periodic* or *aperiodic*
- If we have *m* periodic events and event *i* occurs with period P_i and takes C_i time to process, we need

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

- If this is satisfied we say the real-time system is *schedulable*

Example

- Consider a soft real-time system with 3 periodic events with periods: 100, 200 and 500 msec.
- They require 50, 30 and 100 msec of CPU time respectively
- Are they schedulable?

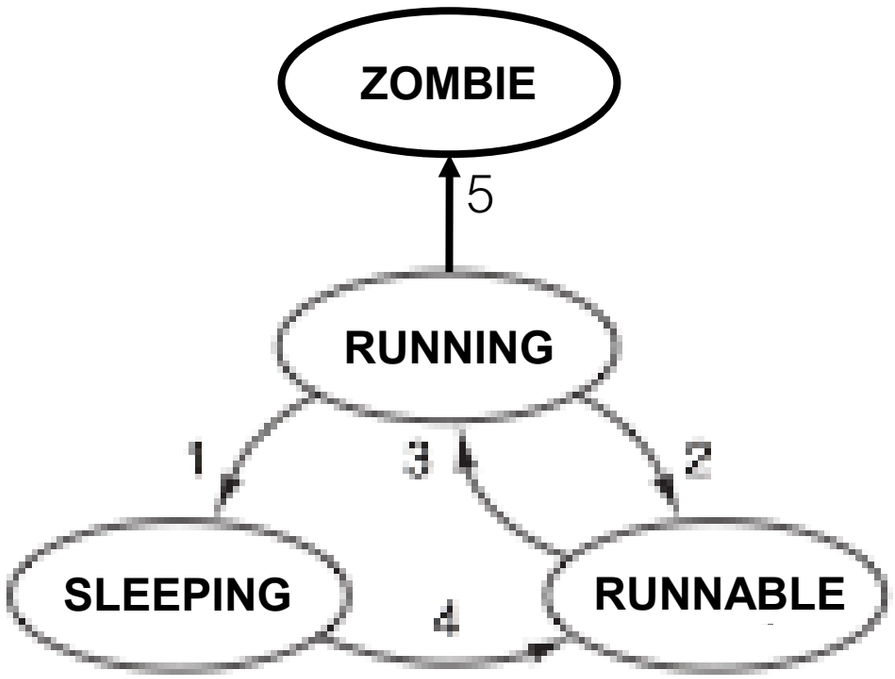
Answer

- 50/100 + 30/200 + 100/500
- .5 + .15 + .2 <= 1
- So yes !! They are schedulable.

Scheduling Algorithms

- Intro
- Scheduling in Batch Systems
- Scheduling in Interactive Systems
- Scheduling in Real-Time Systems
- Scheduling in XV6

xv6: State Transitions



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available
5. **Process Exits**

1. Process Blocks for Input

RUNNING → SLEEPING

- Example: waiting for the user to type something
- In `console.c`, `consoleread` calls **`sleep()`** to put any process waiting for keyboard input to sleep until a key is pressed

```c
int
consoleread(struct inode *ip, char *dst, int n)
{
  uint target;
  int c;

  iunlock(ip);
  target = n;
  acquire(&input.lock);
  while(n > 0){
    while(input.r == input.w){
      if(proc->killed){
        release(&input.lock);
        ilock(ip);
        return -1;
      }
      sleep(&input.r, &input.lock);
    }
```

```
// Atomically release lock and sleep on chan.
// Reacquires lock when awakened.
void
sleep(void *chan, struct spinlock *lk)
{
[... locking code omitted ...]
  // Go to sleep.
  proc->chan = chan;
  proc->state = SLEEPING;
  sched();

  // Tidy up.
  proc->chan = 0;

[... code omitted ...]
}
```

Remember that sched() switches from process back to the kernel

2. Scheduler Picks Another Process

RUNNING → RUNNABLE

- Example: time slice is up (timer interrupt fires)
- In trap.c, the trap function will forcibly give up the CPU on a timer interrupt

Trap

```
void
trap(struct trapframe *tf)
{

[... other trap handling omitted ...]

  // Force process to give up CPU on clock tick.
  // If interrupts were on while locks held, would need to check nlock.
  if(proc && proc->state == RUNNING && tf->trapno == T_IRQ0+IRQ_TIMER)
    yield();

  // Check if the process has been killed since we yielded
  if(proc && proc->killed && (tf->cs&3) == DPL_USER)
    exit();
}
```

Gives up the CPU

Yielding

```
// Give up the CPU for one
//  scheduling round.
void
yield(void)
{
  acquire(&ptable.lock);  //DOC: yieldlock
  proc->state = RUNNABLE;
  sched();
  release(&ptable.lock);
}
```

3. Scheduler Picks This Process

RUNNABLE → **RUNNING**

- Example: the main scheduler() function in proc.c

```
Void scheduler(void)
{
  [ . . . ]
  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;

      // Switch to chosen process.  It is the process's job
      // to release ptable.lock and then reacquire it
      // before jumping back to us.
      proc = p;
      switchuvm(p);
      p->state = RUNNING;
      swtch(&cpu->scheduler, proc->context);
      switchkvm();

      // Process is done running for now.
      // It should have changed its p->state before coming back.
      proc = 0;
    }
    release(&ptable.lock);
  }
}
```

4. Input Becomes Available

SLEEPING ⟶ RUNNABLE

- Example: user presses a key
- In trap.c, the interrupt handler recognizes that a keyboard interrupt has occurred and calls the keyboard handler, which calls the console handler
- Console handler finally calls wakeup() to notify any processes waiting for keyboard input

trap.c

```c
void
trap(struct trapframe *tf)
{
[...]
  switch(tf->trapno){
[...]
  case T_IRQ0 + IRQ_KBD:
    kbdintr();
[...]
  }
}
```

kbd.c

```c
void
kbdintr(void)
{
  consoleintr(kbdgetc);
}
```

console.c

```c
void
consoleintr(int (*getc)(void))
{
[...]
        if(c == '\n' || c == C('D') || input.e == input.r+INPUT_BUF){
          input.w = input.e;
          wakeup(&input.r);
        }
[...]
}
```

proc.c

```c
// Wake up all processes sleeping on chan.
void
wakeup(void *chan)
{
  acquire(&ptable.lock);
  wakeup1(chan);
  release(&ptable.lock);
}

// Wake up all processes sleeping on chan.
// The ptable lock must be held.
static void
wakeup1(void *chan)
{
  struct proc *p;

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == SLEEPING && p->chan == chan)
      p->state = RUNNABLE;
}
```

5. Process Exits

RUNNING → **ZOMBIE**

- Happens at process exit (i.e. the exit() system call)
- Process gets marked as a zombie, and anyone that might be waiting for its exit status gets woken up
- Process doesn't actually get destroyed until someone calls wait()
- If the parent isn't around to call wait(), the zombie gets reparented (assigned to init)

proc.c

```c
// Exit the current process.  Does not return.
// An exited process remains in the zombie state
// until its parent calls wait() to find out it exited.
void
exit(void)
{
  struct proc *p;
[...]
  // Parent might be sleeping in wait().
  wakeup1(proc->parent);

  // Pass abandoned children to init.
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->parent == proc){
      p->parent = initproc;
      if(p->state == ZOMBIE)
        wakeup1(initproc);
    }
  }

  // Jump into the scheduler, never to return.
  proc->state = ZOMBIE;
  sched();
  panic("zombie exit");
}
```

Who Cleans Up?

```c
// Wait for a child process to exit and return its pid.
// Return -1 if this process has no children.
int
wait(void)
{
  struct proc *p;
  int havekids, pid;

  acquire(&ptable.lock);
  for(;;){
    // Scan through table looking for zombie children.
    havekids = 0;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->parent != proc)
        continue;
      havekids = 1;
      if(p->state == ZOMBIE){
        // Found one.
        pid = p->pid;
        kfree(p->kstack);
        p->kstack = 0;
        freevm(p->pgdir);
        p->state = UNUSED;
        p->pid = 0;
        p->parent = 0;
        p->name[0] = 0;
        p->killed = 0;
        release(&ptable.lock);
        return pid;
      }
    }

[...]

    // Wait for children to exit.  (See wakeup1 call in proc_exit.)
    sleep(proc, &ptable.lock);  //DOC: wait-sleep
  }
}
```

The Role of Init

- Init is the first process created
- It spawns the system shell (sh)
- After starting the shell, sits in a loop calling wait() in case any zombies get assigned to it

init.c

```c
int
main(void)
{
  int pid, wpid;
[...]
  for(;;){
[...]
    while((wpid=wait()) >= 0 && wpid != pid)
      printf(1, "zombie!\n");
  }
}
```

Question

- How can we create an orphan zombie that must be adopted by init?
- Or, restated – how do we make init.c reach the printf that prints "zombie!" ?

zombie.c

```c
// Create a zombie process that
// must be reparented at exit.

#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
  if(fork() > 0)
    sleep(5);  // Let child exit before parent.
  exit();
}
```

QEMU

```
SeaBIOS (version rel-1.7.5-0-ge51488c-20140602_164612-nilsson.home.kraxel.org)


iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+1FF93BB0+1FEF3BB0 C980



Booting from Hard Disk...

cpu0: starting xv6

cpu1: starting
cpu0: starting
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ zombie
zombie!
$ _
```